Hannah D'Cruz
Dr. Ming Li
CS 2123
26 Sept 2025

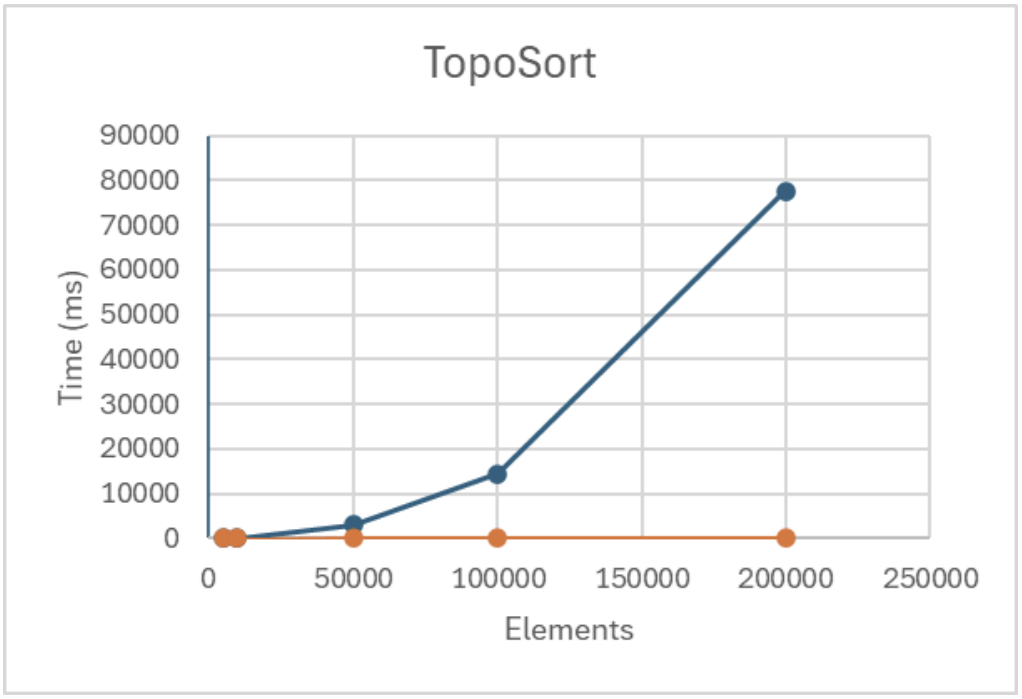
Performance of Topological Sorting Algorithms

The DFS-based implementation of topological sorting is faster than the inductive-based implementation (see Table 1, Figure 1). This is because the running time of the induction-based algorithm is $O(n^2)$, where n is the number of nodes, because for every node it removes it has to then check all the other nodes again to find the next source node. In contrast, the DFS-based algorithm has a running time of O(m + n), where m is the number of edges and n is the number of nodes. This is because it just finds the deepest edge, labels it, and works its way back up the graph once. Since our graph is relatively sparse, with only a few edges for each node, the DFS-based algorithm is even better than the induction-based algorithm. If there were a lot of edges, the DFS algorithm would take longer because it would have to check all the adjacent edges for each node it stops on. The running time of the induction-based algorithm is unchanged by the number of edges, because for each loop and each node, it just checks whether it has any incoming edges or not. Thus, because the number of nodes is the value changing significantly, not the number of edges per node, the DFS-based algorithm is significantly faster than the induction-based algorithm.

This performance difference increases as the number of nodes in the graph increases (see Figure 2). This is because the DFS-based algorithm increases linearly as nodes increase, and the induction-based algorithm increases quadratically as nodes increase, so the derivative of the DFS-based one is constant while the derivative of the induction-based one is still proportional to the number of nodes. The speedup graph is roughly linear because $n^2/(m + n) = n + n/m$, or in terms of the number of nodes it is just a linear n. This means that for larger graphs, it is even more important to use the DFS-based algorithm instead of the induction-based one to improve performance.

Table 1		Inductive	DFS based	
Name	Number of vertices	Average time (ms)	Average time (ms)	Speedup (Ti/Td)
DAG1	5000	75.4	2.6	29
DAG2	10000	104.4	2.4	43.5
DAG3	50000	3009.6	12.4	242.7097
DAG4	100000	14511.4	14.4	1007.736
DAG5	200000	77662.4	24.2	3209.19

Figure 1



Orange: DFS based Blue: Inductive

Figure 2

